

AfarRHB Inventory Management System - Technical Specification

Document Information

Project Name: AfarRHB Inventory App

Hosted URL: <https://afarrhb-inventory.base44.app>

Platform: Base44

Version: 1.0

Last Updated: 2025-11-08

Document Purpose: Complete technical specification for system architecture, implementation, and future development

Executive Summary

The AfarRHB Inventory Management System is a comprehensive enterprise-grade application built for managing inventory, fleet vehicles, requests, issuances, and organizational structures. The system features a modern, gradient-based UI with glass-morphism effects, full CRUD operations, real-time data synchronization, and advanced features like vehicle tracking, geofencing, and audit logging.

Technology Stack

Frontend Framework

- **React 18** - Functional components with Hooks
- **React Router DOM** - Client-side routing
- **TypeScript/JavaScript** - ES6+ syntax

UI Libraries

- **Shadcn UI** - Component library (all components installed)
- **Tailwind CSS** - Utility-first styling
- **Lucide React** - Icon system
- **Framer Motion** - Animations and transitions

State Management & Data Fetching

- **@tanstack/react-query** - Server state management, caching, and synchronization
- **React Hooks** - Local state (useState, useEffect, useRef)

Backend & API

- **Base44 SDK** - Pre-initialized client (@/api/base44Client)
- **Base44 Auth** - Built-in authentication system
- **Base44 Entities API** - Database operations (CRUD)

- **Base44 Integrations** - Core integrations (LLM, Email, File Upload, etc.)

Data Visualization

- **Recharts** - Charts and graphs (Bar, Pie, Area, Line)

Map & Location

- **React Leaflet** - Interactive maps for vehicle tracking and geofencing

Form Handling

- **React Hook Form** - Form validation and management

Other Libraries

- **date-fns** - Date formatting and manipulation
- **lodash** - Utility functions
- **react-markdown** - Markdown rendering
- **react-quill** - Rich text editor
- **@hello-pangea/dnd** - Drag and drop functionality

🎨 Design System

Color Palette

Primary Gradients

- **Violet-Purple-Pink:** from-violet-500 via-purple-500 to-pink-500
- **Emerald:** from-emerald-500 to-emerald-600 (Issuances)
- **Blue:** from-blue-500 to-blue-600 (Items)
- **Indigo:** from-indigo-500 to-indigo-600 (Directorates, Audit Logs)
- **Orange:** from-orange-500 to-orange-600 (Warehouses, Alerts)
- **Green:** from-green-500 to-green-600 (Employees)
- **Purple:** from-purple-500 to-purple-600 (Categories, Requests)
- **Pink:** from-pink-500 to-pink-600 (Customers)

Status Colors

- **Success:** Green (bg-green-100 text-green-800)
- **Warning:** Yellow/Orange (bg-orange-100 text-orange-800)
- **Error:** Red (bg-red-100 text-red-800)
- **Info:** Blue (bg-blue-100 text-blue-800)
- **Neutral:** Gray (bg-gray-100 text-gray-800)

Design Patterns

Glass Morphism

```
.glass-card {
  background: rgba(255, 255, 255, 0.7);
  backdrop-filter: blur(20px);
  border: 1px solid rgba(255, 255, 255, 0.3);
}
```

```
box-shadow: 0 8px 32px 0 rgba(31, 38, 135, 0.15);
}
```

Gradient Backgrounds

- Main content areas: bg-gradient-to-br from-slate-50 via-{color}-50 to-slate-50
- Cards: Glass-morphism with backdrop blur
- Buttons: Gradient hover effects
- Badges: Status-specific colors

Typography

- **Headings:** Bold, gradient text effects
- **Body:** Muted foreground colors
- **Monospace:** For codes, IDs, and technical data

Spacing

- **Consistent:** 4px increments (gap-2, gap-4, gap-6, p-4, p-6, etc.)
- **Cards:** Rounded corners (rounded-xl, rounded-2xl)
- **Responsive:** Grid layouts that collapse on mobile

Project Structure

Directory Layout

/src

/api

base44Client.js # Pre-initialized Base44 SDK

/components

/ui # Shadcn UI components

button.jsx

card.jsx

input.jsx

table.jsx

dialog.jsx

badge.jsx

... (all Shadcn components)

DataTable.jsx # Reusable data table with sorting, filtering, pagination

SearchableSelect.jsx # Searchable entity selector (combobox)

/pages

Dashboard.js

Items.js

ItemView.js

ItemEdit.js
ItemCreate.js
Categories.js
Warehouses.js
Employees.js
Directorates.js
Customers.js
Requests.js
RequestView.js
RequestCreate.js
Issuances.js
IssuanceView.js
IssuanceCreate.js
Vehicles.js
VehicleAssignments.js
VehicleServices.js
VehicleGarages.js
VehicleMap.js
VehicleTracking.js
VehicleAlerts.js
Geofences.js
AuditLogs.js
/entities
(All entity JSON schemas)
/utils
index.js # Utility functions (createPageUrl, etc.)
Layout.js # Main application layout with sidebar

Database Schema (Entities)

1. Item

Primary inventory entity for tracking stock items.

```
{  
  "name": "Item",  
  "properties": {  
    "name": "string (required)",  
    "name_am": "string (Amharic)",  
    "code": "string (required, unique SKU)",
```

```
"category_id": "string (reference to Category)",
"warehouse_id": "string (reference to Warehouse)",
"description": "string",
"unit": "string (required, e.g., pcs, kg, liter)",
"current_stock": "number (default: 0)",
"min_stock_level": "number (default: 0, for alerts)",
"unit_price": "number (default: 0)",
"barcode": "string",
"image_url": "string",
"is_active": "boolean (default: true)"
}
}
```

Built-in Fields: id, created_date, updated_date, created_by

2. Category

Hierarchical categorization for inventory items.

```
{
  "name": "Category",
  "properties": {
    "name": "string (required)",
    "name_am": "string (Amharic)",
    "code": "string (required, unique)",
    "description": "string",
    "parent_category_id": "string (self-reference for hierarchy)",
    "is_active": "boolean (default: true)"
  }
}
```

3. Warehouse

Storage facilities for inventory.

```
{
  "name": "Warehouse",
  "properties": {
    "name": "string (required)",
    "name_am": "string (Amharic)",
    "code": "string (required, unique)",
```

```
"location": "string (address)",
"manager_name": "string",
"phone": "string",
"is_active": "boolean (default: true)"
}
}
```

4. Employee

Employee records with photo support.

```
{
  "name": "Employee",
  "properties": {
    "name": "string (required)",
    "name_am": "string (Amharic)",
    "employee_code": "string (required, unique)",
    "salary": "number",
    "taamagoli": "string",
    "directorate": "string (reference to Directorate)",
    "department": "string",
    "position": "string",
    "phone": "string",
    "email": "string",
    "photo": "string (URL)",
    "is_active": "boolean (default: true)"
  }
}
```

5. Directorate

Organizational units/departments.

```
{
  "name": "Directorate",
  "properties": {
    "name": "string (required)",
    "name_am": "string (Amharic)",
    "code": "string (required, unique)",
    "director_name": "string",

```

```
"phone": "string",
"email": "string",
"parent_directorate_id": "string (hierarchy)",
"is_active": "boolean (default: true)"
}
}
```

6. Customer

External customers/clients.

```
{
  "name": "Customer",
  "properties": {
    "name": "string (required)",
    "name_am": "string (Amharic)",
    "contact_person": "string",
    "phone": "string",
    "email": "string",
    "address": "string",
    "tin_number": "string (Tax ID)",
    "is_active": "boolean (default: true)"
  }
}
```

7. Request

Item requisition workflow.

```
{
  "name": "Request",
  "properties": {
    "request_number": "string (auto-generated)",
    "requester_type": "enum [employee, directorate, customer]",
    "employee_id": "string (conditional)",
    "directorate_id": "string (conditional)",
    "customer_id": "string (conditional)",
    "warehouse_id": "string",
    "request_date": "date (required)",
    "needed_by_date": "date",
  }
}
```

```
"purpose": "string",
"status": "enum [pending, approved, rejected, fulfilled, cancelled]",
"approved_by": "string (email)",
"approved_date": "datetime",
"rejection_reason": "string",
"notes": "string"
}
}
```

8. RequestItem

Line items for requests.

```
{
  "name": "RequestItem",
  "properties": {
    "request_id": "string (required)",
    "item_id": "string (required)",
    "quantity_requested": "number (required)",
    "quantity_approved": "number",
    "notes": "string"
  }
}
```

9. Issuance

Item distribution to recipients.

```
{
  "name": "Issuance",
  "properties": {
    "issuance_number": "string (auto-generated)",
    "request_id": "string (optional reference)",
    "recipient_type": "enum [employee, directorate, customer]",
    "employee_id": "string (conditional)",
    "directorate_id": "string (conditional)",
    "customer_id": "string (conditional)",
    "warehouse_id": "string (required)",
    "issuance_date": "date (required)",
    "issued_by": "string (email, auto-filled)",
  }
}
```



```
"purpose": "string",
"status": "enum [draft, completed, cancelled]",
"notes": "string",
"total_value": "number"
}
}
```

10. Issuanceltem

Line items for issuances.

```
{
  "name": "Issuanceltem",
  "properties": {
    "issuance_id": "string (required)",
    "item_id": "string (required)",
    "quantity": "number (required)",
    "unit_price": "number",
    "total_price": "number",
    "notes": "string"
  }
}
```

11. Vehicle

Fleet vehicle registry.

```
{
  "name": "Vehicle",
  "properties": {
    "vehicle_number": "string (required, plate number)",
    "make": "string (required)",
    "model": "string (required)",
    "year": "number",
    "vin": "string (Vehicle ID Number)",
    "type": "enum [sedan, suv, truck, van, bus, motorcycle, other]",
    "fuel_type": "enum [gasoline, diesel, electric, hybrid]",
    "color": "string",
    "current_mileage": "number (default: 0)",
    "status": "enum [available, assigned, maintenance, out_of_service]",
  }
}
```

```
"purchase_date": "date",
"insurance_expiry": "date",
"next_service_date": "date",
"notes": "string",
"is_active": "boolean (default: true)"
}
}
```

12. VehicleAssignment

Vehicle-to-driver assignments.

```
{
  "name": "VehicleAssignment",
  "properties": {
    "vehicle_id": "string (required)",
    "driver_id": "string (required, Employee reference)",
    "assignment_number": "string (unique)",
    "start_date": "datetime (required)",
    "end_date": "datetime",
    "origin": "string",
    "destination": "string",
    "purpose": "string",
    "starting_mileage": "number",
    "ending_mileage": "number",
    "status": "enum [scheduled, active, completed, cancelled]",
    "notes": "string"
  }
}
```

13. VehicleService

Service and maintenance records.

```
{
  "name": "VehicleService",
  "properties": {
    "vehicle_id": "string (required)",
    "garage_id": "string (VehicleGarage reference)",
    "service_number": "string (unique)",

```

```

    "service_type": "enum [routine_maintenance, repair, inspection, oil_change,
tire_change, other]",
    "description": "string",
    "service_date": "date (required)",
    "completion_date": "date",
    "mileage_at_service": "number",
    "cost": "number (default: 0)",
    "payment_method": "enum [cash, bank_transfer, check, credit]",
    "parts_list": "string",
    "next_service_date": "date",
    "status": "enum [scheduled, in_progress, completed, cancelled]",
    "notes": "string"
  }
}

```

14. VehicleGarage

Service provider registry.

```

{
  "name": "VehicleGarage",
  "properties": {
    "name": "string (required)",
    "name_am": "string (Amharic)",
    "contact_person": "string",
    "phone": "string (required)",
    "email": "string",
    "address": "string",
    "specialization": "string",
    "contract_number": "string",
    "contract_start_date": "date",
    "contract_end_date": "date",
    "is_active": "boolean (default: true)"
  }
}

```

15. MaintenanceSchedule

Automated maintenance scheduling.

```

{
  "name": "MaintenanceSchedule",
  "properties": {
    "vehicle_id": "string (required)",
    "schedule_name": "string (required)",
    "schedule_type": "enum [mileage_based, time_based, both]",
    "mileage_interval": "number (kilometers)",
    "time_interval_months": "number",
    "last_service_date": "date",
    "last_service_mileage": "number",
    "next_service_date": "date (calculated)",
    "next_service_mileage": "number (calculated)",
    "service_type": "enum [routine_maintenance, oil_change, tire_change, inspection, brake_service, other]",
    "description": "string",
    "estimated_cost": "number",
    "alert_before_days": "number (default: 7)",
    "alert_before_mileage": "number (default: 500)",
    "is_recurring": "boolean (default: true)",
    "is_active": "boolean (default: true)"
  }
}

```

16. VehicleTracking

GPS tracking data points.

```

{
  "name": "VehicleTracking",
  "properties": {
    "vehicle_id": "string (required)",
    "assignment_id": "string (optional)",
    "latitude": "number (required)",
    "longitude": "number (required)",
    "timestamp": "datetime (required)",
    "speed": "number (km/h)",
    "heading": "number (degrees)",
    "accuracy": "number (meters)",
    "source": "enum [manual, gps_device, mobile_app, api]",
  }
}

```

```
"note": "string",
"photo_url": "string"
}
}
```

17. Geofence

Geographic boundaries for vehicle monitoring.

```
{
  "name": "Geofence",
  "properties": {
    "name": "string (required)",
    "name_am": "string (Amharic)",
    "type": "enum [circle, polygon]",
    "center_lat": "number (for circles)",
    "center_lng": "number (for circles)",
    "radius_meters": "number (for circles)",
    "polygon_coordinates": "string (JSON array of [lat,lng] pairs)",
    "alert_on_enter": "boolean (default: true)",
    "alert_on_exit": "boolean (default: true)",
    "allowed_vehicles": "string (JSON array of vehicle IDs)",
    "color": "string (default: #3b82f6)",
    "is_active": "boolean (default: true)"
  }
}
```

18. VehicleAlert

Automated alerts and notifications.

```
{
  "name": "VehicleAlert",
  "properties": {
    "vehicle_id": "string (required)",
    "assignment_id": "string (optional)",
    "alert_type": "enum [geofence_enter, geofence_exit, speeding, maintenance_due, insurance_expiry, long_idle, other]",
    "severity": "enum [low, medium, high, critical]",
    "title": "string (required)",

```

```

    "description": "string",
    "geofence_id": "string (if geofence alert)",
    "latitude": "number",
    "longitude": "number",
    "speed": "number",
    "speed_limit": "number",
    "timestamp": "datetime (required)",
    "status": "enum [new, acknowledged, resolved, dismissed]",
    "acknowledged_by": "string (email)",
    "acknowledged_at": "datetime",
    "notes": "string"
  }
}

```

19. AuditLog

System activity tracking.

```

{
  "name": "AuditLog",
  "properties": {
    "user_email": "string (required)",
    "action": "enum [create, update, delete, approve, reject, issue, assign, complete]",
    "entity_type": "string (required)",
    "entity_id": "string",
    "changes": "string (JSON)",
    "ip_address": "string",
    "user_agent": "string",
    "timestamp": "datetime"
  }
}

```

20. Attachment

File attachments for various entities.

```

{
  "name": "Attachment",
  "properties": {
    "entity_type": "string (required)",

```

```

"entity_id": "string (required)",
"file_name": "string (required)",
"file_url": "string (required)",
"file_type": "string (MIME type)",
"file_size": "number (bytes)",
"description": "string"
}
}

```

21. User (Built-in)

Note: This is a built-in Base44 entity. Do not create manually.

```

{
  "name": "User",
  "built_in_properties": {
    "id": "string (auto)",
    "created_date": "datetime (auto)",
    "full_name": "string",
    "email": "string (unique)",
    "role": "enum [admin, user]"
  },
  "custom_properties": {
    "theme": "string (light/dark)",
    "language": "string (en/am)"
  }
}

```

Pages

Core Pages

1. *Dashboard (/Dashboard)*

Purpose: Main landing page with KPIs, charts, and quick actions.

Features:

- 4 KPI cards (Total Items, Low Stock Alerts, Pending Requests, Active Vehicles)
- Sparkline trends (30-day data)
- Quick action buttons (New Item, Create Request, Issue Items, Track Vehicle)
- Issuances trend chart (7-day bar chart)

- Recent issuances list
- Vehicle status distribution (pie chart)
- Recent activity feed
- Low stock alert section

Key Components:

- MetricCard (with sparklines)
- QuickAction cards
- Recharts (BarChart, PieChart, AreaChart)

2. Items (/Items)

Purpose: View and manage inventory items.

Features:

- Stats cards (Total Items, Low Stock, Categories, Total Value)
- Advanced filters (Category, Warehouse, Status, Low Stock)
- DataTable with sorting, search, pagination
- CRUD operations (View, Edit, Delete)
- CSV export
- Low stock highlighting

Columns:

- Image, Code, Name, Category, Stock (with alerts), Price, Status

3. ItemView (/ItemView?id={id})

Purpose: Detailed view of a single item.

Features:

- Large item image display
- Status badge
- Stock level progress bar with alerts
- Basic information (name, code, category, warehouse, barcode)
- Stock & pricing details
- Record metadata (created, updated dates)
- Edit/Delete actions

Layout:

- Left: Image & status
- Right: Detailed information cards

4. ItemCreate (/ItemCreate)

Purpose: Add new inventory items.

Features:

- Image upload with preview
- Form sections: Basic Info, Stock & Pricing, Status
- Real-time validation
- Category and warehouse selection (searchable)
- Auto stock calculation
- Success navigation to ItemView

5. *ItemEdit* (/ItemEdit?id={id})

Purpose: Modify existing items.

Features:

- Pre-populated form
- Image replacement
- Validation
- Stock level updates

Operations Pages**6. *Requests*** (/Requests)

Purpose: Manage item requisitions.

Features:

- 5 status cards (Total, Pending, Approved, Fulfilled, Rejected)
- Status filter dropdown
- DataTable with requester info, warehouse, dates, status
- Approve/Reject inline actions
- Pending requests alert section
- Navigation to RequestView and RequestCreate

7. *RequestView* (/RequestView?id={id})

Purpose: View request details and approve/reject.

Features:

- Status badge and request number
- Requester information
- Requested items table with quantities
- Approval/rejection workflow
- Rejection reason dialog
- "Create Issuance" button for approved requests

8. RequestCreate (/RequestCreate)

Purpose: Create new item requests.

Features:

- Two-column form layout
- Requester type selection (Employee, Directorate, Customer)
- Warehouse selection
- Dynamic item addition with quantity
- Items table preview
- Form validation
- Stock availability checking

9. Issuances (/Issuances)

Purpose: Track item distributions.

Features:

- 4 stats cards (Total, Completed, Draft, Total Value)
- Status filter
- DataTable with recipient info, dates, value, status
- Navigation to IssuanceView and IssuanceCreate
- CSV export

10. IssuanceView (/IssuanceView?id={id})

Purpose: View completed/draft issuance details.

Features:

- Status badge and issuance number
- Two-column layout:
 - Left: Issuance info (date, warehouse, recipient, issuer)
 - Right: Summary (total value, item count, timeline)
- Issued items table with pricing
- Grand total calculation
- Print functionality
- Edit/Delete for drafts

11. IssuanceCreate (/IssuanceCreate)

Purpose: Issue items from inventory.

Features:

- Two-column form:
 - Left: Issuance details (date, warehouse, recipient, purpose)

- Right: Item selection with stock checking
- Dynamic items table
- Real-time total calculation
- Stock deduction on submit
- Validation (recipient, warehouse, items)
- Success navigation to IssuanceView

People & Organization Pages

12. Employees (/Employees)

Purpose: Manage employee records.

Features:

- 4 stats cards (Total, Active, Directorates, Total Salary)
- Photo upload support
- DataTable with photo, code, name, position, directorate, salary, phone, status
- CRUD operations
- Searchable directorate selection

13. Directorates (/Directorates)

Purpose: Manage organizational units.

Features:

- 3 stats cards (Total, Active, Inactive)
- DataTable with code, name, director, phone, email, status
- CRUD operations in modal dialog
- Hierarchical structure support (parent_directorate_id)

14. Customers (/Customers)

Purpose: Manage external clients.

Features:

- 3 stats cards
- DataTable with name, contact person, phone, email, TIN, status
- CRUD operations in modal dialog
- Address and TIN tracking

15. Categories (/Categories)

Purpose: Organize inventory classification.

Features:

- 3 stats cards

- DataTable with code, name, description, status
- CRUD operations in modal dialog
- Hierarchical support

16. Warehouses (/Warehouses)

Purpose: Manage storage facilities.

Features:

- 3 stats cards
- DataTable with code, name, location, manager, phone, status
- CRUD operations in modal dialog

Fleet Management Pages

17. Vehicles (/Vehicles)

Purpose: Manage fleet vehicles.

Features:

- 5 stats cards (Total Fleet, Available, Assigned, Maintenance, Expiring Insurance)
- DataTable with vehicle number, make/model, type, fuel, mileage, status
- CRUD operations in modal dialog
- Status color coding
- Navigation to VehicleMap

18. VehicleAssignments (/VehicleAssignments)

Purpose: Manage vehicle-driver assignments.

Features:

- 4 stats cards
- DataTable with assignment number, vehicle, driver, destination, dates, status
- CRUD operations in modal dialog
- Mileage tracking

19. VehicleServices (/VehicleServices)

Purpose: Track maintenance and repairs.

Features:

- 4 stats cards (Total, Scheduled, Completed, Total Cost)
- DataTable with service number, vehicle, type, date, cost, status
- CRUD operations in modal dialog
- Garage selection
- Parts list tracking

20. *VehicleGarages* (/VehicleGarages)

Purpose: Manage service providers.

Features:

- 3 stats cards
- DataTable with name, contact, phone, specialization, contract expiry, status
- CRUD operations in modal dialog
- Contract tracking

21. *VehicleMap* (/VehicleMap)

Purpose: Real-time vehicle location tracking.

Features:

- Full-screen React Leaflet map
- Custom vehicle markers with status colors
- Clickable popups with vehicle info
- Search and filter by vehicle/status
- Recent alerts sidebar
- Geofence visualization
- Refresh controls
- Stats overlay

22. *VehicleTracking* (/VehicleTracking)

Purpose: Manual check-in/tracking submission.

Features:

- Vehicle selection
- Current location capture (browser geolocation)
- Optional notes and photo upload
- Submit tracking point
- Geofence violation checking
- Speed limit monitoring

23. *VehicleAlerts* (/VehicleAlerts)

Purpose: View and manage vehicle alerts.

Features:

- 4 stats cards (New, Acknowledged, Resolved, Critical)
- Filters by status
- DataTable with severity, type, vehicle, title, description, timestamp, status

- Color-coded severity badges
- Acknowledge and resolve actions
- Resolution notes dialog

24. Geofences (/Geofences)

Purpose: Define geographic boundaries.

Features:

- 3 stats cards
- Interactive map for geofence visualization
- DataTable with name, type, size, alerts, color, status
- CRUD operations in modal dialog
- Circle and polygon support
- Alert configuration (on enter/exit)
- Color picker
- Vehicle whitelist

System Pages

25. AuditLogs (/AuditLogs)

Purpose: System activity monitoring.

Features:

- 5 stats cards (Total, Today, Creates, Updates, Deletes)
- Advanced filters (search, date, action type)
- DataTable with user avatar, action badge, entity type, description, IP, timestamp
- Color-coded action badges
- CSV export
- User initials in avatars

Core Components

1. DataTable (/components/DataTable.jsx)

Purpose: Reusable table with advanced features.

Features:

- Column sorting (ascending/descending)
- Search filtering
- Pagination (configurable page size)
- Action buttons (View, Edit, Delete)
- CSV export
- Loading skeletons

- Empty states
- Delete confirmation dialog

Props:

- columns - Array of column definitions
- data - Table data array
- loading - Loading state
- onView - View callback
- onEdit - Edit callback
- onDelete - Delete callback
- onExport - Export handler
- searchPlaceholder - Search input placeholder
- pageSize - Rows per page (default: 10)

2. **SearchableSelect** (/components/SearchableSelect.jsx)

Purpose: Searchable entity dropdown (Combobox).

Features:

- Real-time entity data fetching
- Search with filtering
- Base44 entity integration
- Customizable label/value fields
- Filter function support
- Loading states

Props:

- entityName - Base44 entity name
- value - Selected value
- onChange - Change handler
- placeholder - Input placeholder
- labelField - Display field (default: "name")
- valueField - Value field (default: "id")
- filterFn - Additional filtering function
- disabled - Disabled state
- className - Custom styles

3. **Layout** (/Layout.js)

Purpose: Main application layout with sidebar navigation.

Features:

Sidebar (Desktop)

- Collapsible design (280px ↔ 80px)
- Smooth Framer Motion animations
- Logo header
- Navigation structure:
 - Single items (Dashboard, Items, Categories, Warehouses)
 - Accordion groups (Operations, People & Organizations, Fleet Management, System)
- User profile card (avatar, name, role)
- Theme toggle (light/dark)
- Language toggle (en/am)
- Logout button
- Collapse/expand toggle
- State persistence (localStorage)

Sidebar (Mobile)

- Slide-in drawer animation
- Touch-friendly spacing
- Backdrop overlay
- Same navigation structure

Active State

- Gradient background for current page
- Icon color changes
- Smooth transitions

Design

- Glass-morphism effect
- Gradient backgrounds
- Rounded corners
- Status-specific coloring

Core Workflows

1. Item Request → Approval → Issuance

1. User creates Request (RequestCreate)

↓

2. Admin views Request (RequestView)

↓

3. Admin approves/rejects Request

↓ (if approved)

4. Admin creates Issuance from Request (IssuanceCreate)

↓

5. System deducts stock automatically

↓

6. User views Issuance details (IssuanceView)

Entities Involved:

- Request, RequestItem
- Issuance, IssuanceItem
- Item (stock update)
- User (permissions)

2. Low Stock Alert → Restock

1. System detects $\text{item.current_stock} \leq \text{item.min_stock_level}$

↓

2. Dashboard shows Low Stock Alert card

↓

3. User clicks alert → navigates to Items page

↓

4. User edits Item (ItemEdit)

↓

5. User increases current_stock

↓

6. Alert disappears on next refresh

3. Vehicle Assignment → Tracking → Service

1. Admin assigns vehicle to driver (VehicleAssignments)

↓

2. Driver submits tracking points (VehicleTracking)

↓

3. System monitors geofences and speed

↓ (if violation)

4. System creates VehicleAlert

↓

5. Admin views alert (VehicleAlerts)

↓

6. Admin schedules service (VehicleServices)

↓

7. Vehicle status → "maintenance"

↓

8. Service completed → status → "available"

Entities Involved:

- VehicleAssignment
- VehicleTracking
- Geofence
- VehicleAlert
- VehicleService
- Vehicle (status updates)

4. Employee Photo Upload

1. Admin navigates to Employees page

↓

2. Admin clicks "Add Employee" or "Edit"

↓

3. Admin selects photo file

↓

4. Component validates file (size < 5MB, image type)

↓

5. Component uploads file via Base44.integrations.Core.UploadFile

↓

6. Component receives file_url

↓

7. Component saves Employee with photo URL

↓

8. Photo displays in DataTable and view pages

Authentication & Permissions

Built-in Base44 Auth

- **Login:** Handled automatically by Base44 platform
- **User Roles:** admin, user
- **User Object:** Accessible via base44.auth.me()
- **Logout:** base44.auth.logout()

- **Redirect to Login:** `base44.auth.redirectToLogin()`
- **Check Authentication:** `base44.auth.isAuthenticated()`

Security Rules (Built-in for User Entity)

- Admin users can list, update, delete all users
- Regular users can only view/update their own record
- Enforced automatically, cannot be overridden

Custom Permissions (Future Enhancement)

- Role-based access control (RBAC)
- Entity-level permissions
- Field-level visibility
- Action restrictions (e.g., only admins can delete)

Data Flow & State Management

React Query Pattern

// Fetching Data

```
const { data, isLoading, error } = useQuery({
  queryKey: ['entity', params],
  queryFn: () => base44.entities.Entity.list(),
  initialData: [],
});
```

// Creating Data

```
const createMutation = useMutation({
  mutationFn: (data) => base44.entities.Entity.create(data),
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['entity'] });
  },
});
```

// Updating Data

```
const updateMutation = useMutation({
  mutationFn: ({ id, data }) => base44.entities.Entity.update(id, data),
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['entity'] });
  },
});
```

```
// Deleting Data
const deleteMutation = useMutation({
  mutationFn: (id) => base44.entities.Entity.delete(id),
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['entity'] });
  },
});
```

Query Invalidation Strategy

- Invalidate parent queries on create/update/delete
- Invalidate related queries (e.g., invalidate items after issuance)
- Use query keys for granular control

Loading States

- Skeleton loaders for tables and cards
- Spinner buttons during mutations
- Optimistic UI updates (future enhancement)

Key Features

1. Stock Management

- Real-time stock tracking
- Low stock alerts on Dashboard
- Automatic stock deduction on issuance
- Stock history (future enhancement)

2. Request-Approval Workflow

- Multi-step process (request → approve → issue)
- Rejection with reasons
- Approval tracking (who, when)
- Status badges

3. Vehicle Fleet Management

- Real-time GPS tracking
- Geofencing with alerts
- Maintenance scheduling (mileage and time-based)
- Service history
- Assignment tracking

4. Multi-language Support

- English and Amharic (አማርኛ)
- User preference stored in User entity

- Toggle in Layout sidebar
- Applied to labels, buttons, and entity names

5. Dark Mode

- Theme toggle in Layout
- CSS variables for colors
- Persisted in User entity
- Smooth transitions

6. Responsive Design

- Mobile-first approach
- Collapsible sidebar on desktop
- Slide-in drawer on mobile
- Touch-friendly controls
- Grid layouts that stack on small screens

7. Data Export

- CSV export in DataTable component
- Configurable columns
- UTF-8 encoding for Amharic text

8. Audit Logging

- Track all CRUD operations
- User, action, entity type, timestamp
- IP address and user agent tracking
- Filter by date, user, action

9. Photo Uploads

- Employee photos
- Item images
- Tracking point photos (vehicles)
- File size validation (5MB limit)
- Preview before upload

10. Advanced Filtering

- Category, warehouse, status filters
- Date range pickers
- Search across multiple fields
- Real-time filter updates

Known Issues & Limitations

Current Issues

1. **No Mock Data Cleanup:** Dashboard uses mock data for issuances and activity (should fetch from actual entities)
2. **No Pagination on Large Lists:** Some entity lists load all records (500 limit)
3. **No Real-time Updates:** Requires manual refresh for new data
4. **No Offline Support:** Requires internet connection
5. **Limited Vehicle Tracking:** Manual submission only (no GPS device integration)

Missing Features

1. **Reports & Analytics:** No built-in reporting system
2. **Email Notifications:** No automated alerts
3. **Barcode Scanning:** No mobile barcode input
4. **Bulk Operations:** No bulk import/export (except CSV)
5. **Stock History:** No historical stock level tracking
6. **Advanced Permissions:** Single admin/user role only
7. **Print Templates:** Basic browser print only
8. **API Documentation:** No external API exposure
9. **Multi-warehouse Stock:** Items linked to single warehouse only
10. **Vehicle Telemetry:** No real-time GPS feed

Future Enhancements

Priority 1 (MVP+)

1. **Replace Dashboard Mock Data:** Fetch real issuances and activity from database
2. **Email Notifications:** Send alerts for low stock, approvals, rejections
3. **Advanced Search:** Full-text search across all entities
4. **Stock History Tracking:** Log all stock changes with reasons
5. **Barcode Scanning:** Mobile camera integration

Priority 2 (Enterprise)

1. **Reports Module:** Generate PDF reports (stock, issuances, vehicles)
2. **Role-based Permissions:** Custom roles (manager, warehouse_staff, driver)
3. **Bulk Import:** Excel/CSV import for items, employees, vehicles
4. **Automated Maintenance Alerts:** Email/SMS for upcoming services
5. **Real-time GPS Integration:** Connect to GPS devices (future-proof for third-party APIs)
6. **Vehicle Telemetry Dashboard:** Live speed, fuel, engine data
7. **Mobile App:** React Native app for drivers and warehouse staff
8. **Multi-currency Support:** Handle different currencies

9. **Multi-warehouse Stock:** Track items across multiple warehouses
10. **Approval Workflows:** Multi-level approvals with configurable rules

Priority 3 (Advanced)

1. **AI-powered Insights:** Predict stock needs, maintenance schedules
2. **Blockchain Audit Trail:** Immutable audit logs
3. **IoT Integration:** Smart shelves, RFID tags
4. **Voice Commands:** Alexa/Google Home integration
5. **AR Warehouse Navigation:** Augmented reality for locating items

Development Guidelines

Code Style

- **Functional Components:** Use React Hooks (no class components)
- **Naming Conventions:**
 - Components: PascalCase (e.g., DataTable.jsx)
 - Functions: camelCase (e.g., handleSubmit)
 - Constants: UPPER_SNAKE_CASE (e.g., MAX_FILE_SIZE)
- **File Organization:**
 - One component per file
 - Co-locate related components in folders
 - Use index files for barrel exports

Entity Guidelines

- **Always Use Required Fields:** Mark critical fields as required
- **Validation:** Client-side validation + server-side (Base44 handles)
- **Soft Deletes:** Use is_active flag instead of hard deletes
- **Timestamps:** Rely on built-in created_date, updated_date
- **Audit Trail:** Use created_by for user tracking

UI Guidelines

- **Consistent Gradients:** Use theme-specific gradients per module
- **Responsive:** Test on mobile, tablet, desktop
- **Accessibility:** Use semantic HTML, ARIA labels
- **Loading States:** Always show skeleton loaders
- **Error Handling:** Display user-friendly error messages
- **Empty States:** Provide helpful messages and actions

Performance

- **Lazy Loading:** Use React.lazy for code splitting (future)
- **Pagination:** Implement for large datasets
- **Caching:** Leverage React Query's built-in caching

- **Debouncing:** Use for search inputs (future)
- **Image Optimization:** Compress uploads before sending

Testing Strategy (Future)

Unit Tests

- Test utility functions
- Test custom hooks
- Test component logic

Integration Tests

- Test form submissions
- Test API interactions
- Test navigation flows

E2E Tests

- Test critical workflows (request → approval → issuance)
- Test authentication flows
- Test CRUD operations

Tools

- **Jest** - Unit testing
- **React Testing Library** - Component testing
- **Cypress** - E2E testing

Deployment

Base44 Platform Hosting

- **URL:** <https://afarrhb-inventory.base44.app>
- **Build:** Vite bundler
- **Mode:** Production
- **CDN:** Automatic asset optimization
- **SSL:** Enabled by default

Environment Variables

- **VITE_BASE44_API_URL:** Base44 API endpoint (auto-configured)
- **VITE_MAPBOX_TOKEN:** Mapbox API token (for maps)

Build Command

npm run build

Preview Command

npm run preview

Documentation & Resources

Internal Documentation

- **Entity Schemas:** /entities/*.json
- **Component Props:** JSDoc comments in component files
- **API Usage:** See Base44 SDK documentation

External Resources

- **Base44 Platform:** <https://base44.app>
- **Shadcn UI:** <https://ui.shadcn.com>
- **Tailwind CSS:** <https://tailwindcss.com>
- **React Query:** <https://tanstack.com/query>
- **Lucide Icons:** <https://lucide.dev>
- **Recharts:** <https://recharts.org>
- **React Leaflet:** <https://react-leaflet.js.org>

AI Builder Instructions

If you're an AI assistant tasked with replicating or extending this system:

1. **Start with Entities:** Define all 21 entities first
2. **Build Layout:** Create the collapsible sidebar layout
3. **Core Components:** Implement DataTable and SearchableSelect
4. **Critical Pages First:** Dashboard, Items, Requests, Issuances
5. **Follow Design System:** Use gradients, glass-morphism, consistent spacing
6. **Use React Query:** For all data fetching and mutations
7. **Test Workflows:** Request → Approval → Issuance flow is critical
8. **Mobile Responsive:** Test all pages on mobile
9. **Replace Mock Data:** Dashboard mock issuances should fetch from real entities
10. **Validate Thoroughly:** Especially stock deductions and approvals

Support & Maintenance

Contact Information

- **Developer:** Base44 AI Assistant
- **Platform Support:** Base44 Team
- **Feedback:** Use in-app feedback button

Maintenance Schedule

- **Daily:** Monitor error logs
- **Weekly:** Review audit logs for anomalies
- **Monthly:** Database cleanup (soft-deleted records)
- **Quarterly:** Performance optimization

Change Log

Version 1.0 (2025-11-08)

- Initial system specification
- All 25 pages implemented
- 21 entities defined
- Modern gradient UI with glass-morphism
- Collapsible sidebar with Framer Motion
- Vehicle fleet management module
- Audit logging system
- Multi-language support (English, Amharic)
- Dark mode theming

Conclusion

This technical specification provides a comprehensive blueprint for the AfarRHB Inventory Management System. It covers every aspect from database schema to UI design, workflows to future enhancements. Use this document as a single source of truth for development, maintenance, and future AI-assisted code generation.

Key Takeaways:

- 25 fully functional pages
- 21 interconnected entities
- Modern, responsive, gradient-based UI
- Complete CRUD workflows
- Advanced fleet management
- Audit logging and security
- Extensible architecture for future features

This system is production-ready with room for enterprise-grade enhancements. 🚀

END OF SPECIFICATION DOCUMENT